Crio.Do

LEARNING ROADMAP

Services to Product Company Transition

Your Complete Roadmap from TCS/Infosys/Wipro to Google/Amazon/Microsoft

Intensive	Balanced	Flexible
25-30 hrs/week	15-20 hrs/week	8-10 hrs/week

▪ 8 Phases ★ Intermediate to Advanced

https://crio.do

Overview

This battle-tested roadmap is specifically designed for engineers working in Indian IT services companies (TCS, Infosys, Wipro, Cognizant, HCL, Tech Mahindra, etc.) who aspire to break into product-based companies. The transition requires a fundamental shift in how you approach problem-solving, coding, and interviews. This guide addresses the unique challenges services engineers face: limited hands-on coding, project-centric work, and gaps in DSA fundamentals. With structured preparation, thousands have made this transition successfully - and so can you.

Who is this for?

Software engineers currently working in IT services companies with 1-8 years of experience, looking to transition to product-based companies for better compensation, challenging work, and career growth.

Prerequisites

- Currently working as a software engineer (any technology stack)
- Basic programming knowledge in at least one language
- Willingness to dedicate 2-4 hours daily for preparation
- Access to a computer for coding practice
- Patience and persistence - this is a marathon, not a sprint

What you'll learn

- Crack DSA interviews at FAANG and top product companies
- Design scalable systems for system design rounds
- Build an impressive portfolio with real projects
- Master behavioral interviews with structured storytelling
- Develop a product-company mindset and communication style
- Successfully negotiate offers worth 2-4x your current CTC

Learning Plans

Choose a plan that fits your schedule. All plans cover the same content.

Intensive

Fast Track

25-30 hrs/week

500-600 total hrs

Aggressive preparation while working. Requires strong discipline and possibly reducing work commitments. Best for those with urgent timelines or upcoming interviews.

Ideal for:

- Engineers on notice period
- Those with interview calls already scheduled
- People willing to take a break from work
- Highly disciplined self-learners

Balanced

Recommended

15-20 hrs/week

480-640 total hrs

The recommended path for working professionals. Sustainable pace that balances job responsibilities with serious preparation. Most successful transitions happen on this track.

Ideal for:

- Full-time working professionals
- Engineers with 2-3 hours daily available
- Those who want thorough preparation
- People targeting offers within 6-8 months

Flexible

Self-paced

8-10 hrs/week

400-520 total hrs

Flexible schedule for those with heavy work commitments or family responsibilities. Slower but steady progress with adequate time for concept internalization.

Ideal for:

- Engineers with demanding projects
- Parents and caregivers
- Those learning alongside other commitments
- People targeting offers within 10-12 months

Phase Schedule by Plan

Phase	Topic	Intensive	Balanced	Flexible
1	Foundation Reset & Mindset Shift	Week 1	Weeks 1-2	Weeks 1-4
2	DSA Fundamentals	Weeks 2-6	Weeks 3-10	Weeks 5-16
3	DSA Advanced	Weeks 7-12	Weeks 11-20	Weeks 17-32
4	Low Level Design (LLD)	Weeks 13-15	Weeks 21-26	Weeks 33-42
5	System Design (HLD)	Weeks 16-18	Weeks 27-32	Weeks 43-52
6	CS Fundamentals & Projects	Weeks 19-21	Weeks 33-38	Weeks 53-64
7	Behavioral & Interview Mastery	Weeks 22-24	Weeks 39-44	Weeks 65-76
8	Job Search & Continuous Practice	Weeks 25-30	Weeks 45-52	Weeks 77-90

1 Foundation Reset & Mindset Shift

Intensive: Week 1 | Balanced: Weeks 1-2 | Flexible: Weeks 1-4

Before diving into technical preparation, you need to understand what product companies look for and reset your approach to software engineering. Services companies optimize for delivery; product companies optimize for problem-solving, ownership, and scalability.

Learning Objectives

- Understand the key differences between services and product companies
- Choose your primary programming language for interviews
- Set up your coding environment and practice platforms
- Audit your current skills and identify gaps
- Create a LinkedIn profile optimized for product roles
- Establish daily coding habits

Topics to Cover

Services vs Product Mindset

Understanding what product companies value

- Ownership vs Delivery mindset
- Code quality expectations
- Product thinking
- Engineering culture differences
- Compensation structures (base vs RSU vs ESOP)

Language Selection

Choosing the right interview language

- Java vs Python vs C++
- Language-specific trade-offs
- Standard library mastery
- Time complexity awareness
- Clean code principles

Environment Setup

Tools for effective practice

- IDE setup (VS Code/IntelliJ)
- LeetCode account setup
- GitHub profile optimization
- Local development environment
- Time tracking tools

Profile Building

Creating your professional brand

- LinkedIn optimization
- Resume tailoring
- GitHub contributions
- Removing services jargon
- Highlighting impact over tasks

Hands-on Projects

Skills Audit Document

Create a comprehensive self-assessment documenting your current DSA knowledge, system design understanding, project experience, and gap areas. This becomes your preparation compass.

Self-assessment | Goal setting | Planning

LinkedIn & Resume Revamp

Transform your services-style resume and LinkedIn into product-company friendly versions. Focus on impact metrics, technologies used, and problems solved rather than project descriptions.

Personal branding | Communication | Resume writing

Resources

PLATFORM **Crio.Do - Learn by Building**

PLATFORM **LeetCode - Coding Practice Platform**

DOCUMENTATION **GitHub Profile README Guide**

Milestone

Complete skills audit, choose your interview language, solve 10 easy LeetCode problems, and update your LinkedIn profile

2 DSA Fundamentals

Intensive: Weeks 2-6 | Balanced: Weeks 3-10 | Flexible: Weeks 5-16

Data Structures and Algorithms form the core of product company interviews. This phase builds your foundation from the ground up. Unlike services work, you need to internalize these patterns deeply - interviewers expect you to solve novel problems, not memorize solutions.

Learning Objectives

- Master fundamental data structures and their operations
- Understand time and space complexity analysis
- Solve problems using arrays, strings, and hashmaps
- Implement linked lists, stacks, and queues from scratch
- Learn recursion and basic backtracking
- Develop problem-solving intuition

Topics to Cover

Big-O Analysis

Complexity analysis fundamentals

- Time complexity
- Space complexity
- Best/Average/Worst case
- Amortized analysis
- Analyzing loops and recursion

Arrays & Strings

The most common interview topics

- Two pointers technique
- Sliding window
- Prefix sums
- String manipulation
- Subarray problems

Hashmaps & Sets

O(1) lookup patterns

- Frequency counting
- Two sum patterns
- Anagram problems
- Subarray sum equals K
- Design hashmap

Linked Lists

Pointer manipulation mastery

- Singly vs Doubly linked
- Fast and slow pointers
- Reversal techniques
- Merge operations
- Cycle detection

Stacks & Queues

LIFO and FIFO patterns

- Monotonic stack
- Next greater element
- Valid parentheses
- Implementing with arrays
- BFS with queues

Recursion Basics

Foundation for advanced algorithms

- Recursive thinking
- Base cases
- Recursion tree visualization
- Memoization intro
- Backtracking basics

Hands-on Projects

LeetCode 75 - Easy Problems

Complete all easy problems from the LeetCode 75 study plan. Focus on understanding patterns, not just getting accepted answers. Time yourself and aim for optimal solutions.

Arrays | Strings | Hashmaps | Two Pointers | Basic Problem Solving

Implement Data Structures from Scratch

Build ArrayList, LinkedList, HashMap, Stack, and Queue from scratch in your chosen language. This deepens understanding of how these structures work internally.

Implementation | Memory management | API design | Testing

Resources

ROADMAP **NeetCode 150 Roadmap**

REFERENCE **Big-O Cheat Sheet**

VISUALIZATION **Visualgo - Visualize Data Structures**

REPOSITORY **LeetCode Patterns GitHub**

Milestone

Solve 80+ LeetCode problems (60 Easy, 20 Medium) with at least 70% solve rate without hints

3 DSA Advanced

Intensive: Weeks 7-12 | Balanced: Weeks 11-20 | Flexible: Weeks 17-32

Level up to medium and hard problems. This phase covers trees, graphs, dynamic programming, and advanced techniques that separate good candidates from great ones. Product companies test these heavily for SDE-1 and SDE-2 roles.

Learning Objectives

- Master binary trees and binary search trees
- Understand graph representations and traversals
- Solve dynamic programming problems systematically
- Apply binary search to complex scenarios
- Learn heap operations and priority queues
- Handle greedy algorithm problems

Topics to Cover

Trees

Hierarchical data structures

- Binary tree traversals
- BST operations
- Tree construction
- Lowest Common Ancestor
- Tree DP
- Tries

Graphs

Connected components and paths

- BFS and DFS
- Cycle detection
- Topological sort
- Shortest paths (Dijkstra)
- Union-Find
- Minimum spanning tree

Dynamic Programming

The most feared interview topic

- 1D DP patterns
- 2D DP patterns
- Knapsack variations
- LIS and LCS
- DP on strings
- State machine DP

Binary Search Advanced

Beyond simple search

- Search space reduction
- Minimize/Maximize problems
- Rotated arrays
- Search in matrix
- Capacity to ship packages

Heaps & Priority Queues

Top K patterns

- Min and Max heaps
- K largest/smallest
- Merge K sorted
- Meeting rooms
- Median from stream

Greedy Algorithms

Local optimal to global optimal

- Activity selection
- Interval scheduling
- Jump game
- Greedy vs DP decision
- Proof of correctness

Hands-on Projects

NeetCode 150 Completion

Complete the NeetCode 150 problem set covering all major patterns. Document your approach for each problem category. This is the gold standard for interview preparation.

All DSA patterns | Problem categorization | Pattern recognition

LeetCode Contest Participation

Participate in 8-10 weekly LeetCode contests. This builds time pressure handling and exposes you to novel problems. Aim to solve 2-3 problems consistently.

Time management | Problem selection | Performance under pressure

Resources

VIDEO **NeetCode YouTube Channel**

VIDEO **Abdul Bari - Algorithms**

VIDEO **Tech Dose - DP Patterns**

REFERENCE **CP Algorithms**

Milestone

Solve 150+ problems (50 Easy, 80 Medium, 20 Hard) and participate in 10 contests with average rank in top 50%

4 Low Level Design (LLD)

Intensive: Weeks 13-15 | Balanced: Weeks 21-26 | Flexible: Weeks 33-42

Product companies test your ability to write clean, maintainable, and extensible code. Low Level Design rounds assess OOP principles, design patterns, and code organization. This is where services engineers often struggle due to limited greenfield project exposure.

Learning Objectives

- Master Object-Oriented Programming principles
- Apply SOLID principles in design
- Implement common design patterns
- Design class diagrams for real systems
- Write clean, testable code
- Handle concurrency basics

Topics to Cover

OOP Deep Dive

Object-oriented fundamentals

- Encapsulation
- Inheritance vs Composition
- Polymorphism
- Abstraction
- Interface segregation

SOLID Principles

Foundation of good design

- Single Responsibility
- Open-Closed
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

Creational Patterns

Object creation mechanisms

- Singleton
- Factory Method
- Abstract Factory
- Builder
- Prototype

Structural Patterns

Object composition

- Adapter
- Decorator
- Facade
- Composite
- Proxy

Behavioral Patterns

Object communication

- Strategy
- Observer
- Command
- State
- Chain of Responsibility

Clean Code & Testing

Writing maintainable code

- Naming conventions
- Function design
- Code smells
- Unit testing
- Dependency injection

Hands-on Projects

Parking Lot System

Design and implement a parking lot management system with multiple floors, vehicle types, pricing strategies, and entry/exit management. Classic LLD interview question.

OOP | Strategy Pattern | Factory Pattern | State management

Splitwise Clone

Build a simplified expense sharing application with group management, expense splitting (equal, exact, percentage), and balance settlement.

Complex domain modeling | Observer Pattern | Clean architecture

Library Management System

Design a library system with book management, member registration, book lending, fine calculation, and reservation system.

SOLID principles | Decorator Pattern | State Pattern

Resources

REFERENCE **Refactoring Guru - Design Patterns**

REPOSITORY **LLD Problems GitHub Repository**

BOOK **Clean Code by Robert Martin**

BOOK **Head First Design Patterns**

Milestone

Implement 5 LLD problems from scratch with clean code, proper patterns, and unit tests

5 System Design (HLD)

Intensive: Weeks 16-18 | Balanced: Weeks 27-32 | Flexible: Weeks 43-52

For engineers with 2+ years experience, system design is a crucial interview round. This phase teaches you to design scalable distributed systems - something services projects rarely expose you to. Even if you're junior, understanding these concepts makes you a better engineer.

Learning Objectives

- Understand distributed system fundamentals
- Design scalable architectures for real products
- Make informed trade-off decisions
- Calculate capacity and estimate resources
- Communicate designs clearly in interviews
- Handle follow-up questions confidently

Topics to Cover

Fundamentals

Building blocks of distributed systems

- Client-server architecture
- DNS and CDN
- Load balancing
- Caching strategies
- Database types
- API design (REST/GraphQL)

Scalability

Handling growth

- Horizontal vs Vertical scaling
- Database sharding
- Replication
- Consistent hashing
- CAP theorem
- Eventual consistency

Storage

Data persistence patterns

- SQL vs NoSQL
- Database indexing
- Partitioning strategies
- Object storage
- Data warehousing
- Time-series databases

Communication

Inter-service communication

- Synchronous vs Async
- Message queues (Kafka/RabbitMQ)
- Pub-Sub patterns
- WebSockets
- gRPC
- Event-driven architecture

Reliability

Building resilient systems

- Single point of failure
- Redundancy
- Circuit breakers
- Rate limiting
- Monitoring and alerting
- Chaos engineering basics

Common Designs

Must-know system designs

- URL Shortener
- Twitter/Instagram feed
- Chat application
- Notification system
- Rate limiter
- Search autocomplete

Hands-on Projects

Design Document Portfolio

Write detailed design documents for 6-8 common systems (URL shortener, Twitter, Netflix, Uber). Include diagrams, capacity estimates, API designs, and trade-off discussions.

Documentation | Diagramming | Capacity estimation | Trade-off analysis

Mini Distributed System

Build a simplified URL shortener or rate limiter with actual implementation. Use Redis for caching, implement basic load balancing, and deploy to cloud.

Hands-on system design | Cloud deployment | Redis | Docker basics

Resources

REPOSITORY **System Design Primer GitHub**

VIDEO **Gaurav Sen System Design**

VIDEO **ByteByteGo YouTube**

BOOK **Designing Data-Intensive Applications**

BLOG **High Scalability Blog**

Milestone

Complete design documents for 8 systems and successfully explain any design in a 45-minute mock interview

6 CS Fundamentals & Projects

Intensive: Weeks 19-21 | Balanced: Weeks 33-38 | Flexible: Weeks 53-64

Product companies expect solid computer science fundamentals and real project experience. This phase fills gaps in OS, DBMS, and networking while building impressive portfolio projects that demonstrate your skills beyond services work.

Learning Objectives

- Revise Operating System concepts
- Master database fundamentals and SQL
- Understand computer networking basics
- Build 2-3 impressive portfolio projects
- Deploy projects with proper CI/CD
- Write clean documentation

Topics to Cover

Operating Systems

Core OS concepts for interviews

- Process vs Thread
- CPU scheduling
- Memory management
- Virtual memory
- Deadlocks
- Synchronization primitives

Database Concepts

RDBMS fundamentals

- ACID properties
- Normalization
- Indexing
- Transactions
- Joins optimization
- SQL queries

Computer Networks

Networking essentials

- OSI/TCP-IP model
- HTTP/HTTPS
- TCP vs UDP
- DNS resolution
- REST API design
- WebSockets

Version Control

Git mastery

- Branching strategies
- Merge vs Rebase
- Code review best practices
- Git workflows
- Handling conflicts

Project Building

Portfolio development

- Project selection
- Tech stack decisions
- MVP approach
- Documentation
- Deployment
- Demo preparation

Hands-on Projects

Full-Stack Web Application

Build a complete web application (e.g., expense tracker, task manager, or booking system) with authentication, database, REST APIs, and modern frontend. Deploy on cloud with CI/CD.

Full-stack development | Database design | Authentication | Deployment | CI/CD

Backend Microservice

Create a scalable backend service (e.g., URL shortener, rate limiter, or notification service) with proper logging, monitoring, and containerization.

Microservices | Docker | Redis/MongoDB | API design | Logging

Open Source Contribution

Make meaningful contributions to 2-3 open source projects. Start with documentation or bug fixes, then progress to feature additions. This demonstrates collaboration skills.

Open source | Code review | Collaboration | Reading others' code

Resources

PROJECTS **Crio.Do Developer Projects**

DOCUMENTATION **Git Documentation**

DOCUMENTATION **Docker Get Started**

TUTORIAL **PostgreSQL Tutorial**

REPOSITORY **First Contributions GitHub**

Milestone

Deploy 2 portfolio projects with documentation, make 3+ open source contributions, and create comprehensive GitHub profile

7 Behavioral & Interview Mastery

Intensive: Weeks 22-24 | Balanced: Weeks 39-44 | Flexible: Weeks 65-76

Technical skills get you to the interview, but behavioral rounds and communication skills get you the offer. Product companies heavily evaluate cultural fit, leadership potential, and how you handle ambiguity. This phase transforms how you present yourself.

Learning Objectives

- Master the STAR format for behavioral questions
- Prepare compelling stories from your experience
- Practice technical communication skills
- Learn company-specific preparation strategies
- Build a referral network
- Handle salary negotiations confidently

Topics to Cover

Behavioral Interviews

Telling your story effectively

- STAR format
- Leadership stories
- Conflict resolution
- Failure and learning
- Amazon Leadership Principles
- Google Googliness

Technical Communication

Explaining while coding

- Thinking out loud
- Clarifying requirements
- Trade-off discussions
- Handling hints
- Time management

Mock Interviews

Practice makes perfect

- Peer practice sessions
- Platform mock interviews
- Recording self-practice
- Feedback incorporation
- Interview day simulation

Company Research

Targeted preparation

- Company culture
- Team understanding
- Product knowledge
- Recent news
- Interview format research

Networking & Referrals

Getting your foot in the door

- LinkedIn networking
- Referral requests
- Cold outreach
- Informational interviews
- Tech community engagement

Negotiation

Getting the best offer

- Understanding compensation components
- Market research
- Counter-offer strategies
- Multiple offer handling
- Joining bonus negotiation

Hands-on Projects

STAR Stories Document

Prepare 8-10 detailed STAR format stories covering leadership, conflict, failure, success, and impact. Each story should be adaptable to multiple behavioral questions.

Self-reflection | Storytelling | Communication

Mock Interview Marathon

Complete 15+ mock interviews (mix of DSA, system design, and behavioral) with peers or platforms. Document feedback and improvement areas after each session.

Interview performance | Feedback handling | Continuous improvement

Target Company Research

Create detailed research documents for your top 5 target companies covering interview process, team culture, recent products, and potential questions.

Research | Preparation | Company knowledge

Resources

PLATFORM **Pramp - Free Mock Interviews**

PLATFORM **Interviewing.io**

REFERENCE **Amazon Leadership Principles**

REFERENCE **Levels.fyi - Salary Data**

REFERENCE **Glassdoor Interview Reviews**

Milestone

Complete 15 mock interviews with 80%+ positive feedback, prepare stories for all common behavioral questions, and have referrals lined up for 3+ companies

8 Job Search & Continuous Practice

Intensive: Weeks 25-30 | Balanced: Weeks 45-52 | Flexible: Weeks 77-90

The final phase focuses on active job hunting while maintaining interview readiness. Apply strategically, track your progress, and keep practicing until you land your dream offer. Most successful transitions require 3-6 months of active applications.

Learning Objectives

- Apply strategically to target companies
- Maintain daily coding practice
- Track and optimize application process
- Handle multiple interviews simultaneously
- Make informed decisions on offers
- Plan successful onboarding

Topics to Cover

Application Strategy

Smart job hunting

- Company tiering
- Application tracking
- Referral prioritization
- Direct applications
- Recruiter outreach

Interview Scheduling

Managing the process

- Timeline optimization
- Parallel processing
- Company stacking
- Deadline management
- Communication etiquette

Maintaining Readiness

Staying sharp

- Daily practice routine
- Weak area focus
- Contest participation
- New problem exposure
- Review sessions

Offer Evaluation

Making the right choice

- Total compensation calculation
- RSU vesting schedules
- Team evaluation
- Growth potential
- Work-life balance

Transition Planning

Setting up for success

- Notice period management
- Knowledge transfer
- New role preparation
- First 90 days planning
- Building relationships

Hands-on Projects

Job Application Tracker

Create a systematic tracker for all applications including company, role, status, contacts, interview dates, and outcomes. Review weekly to identify patterns and optimize strategy.

Organization | Data analysis | Process optimization

Daily Practice Log

Maintain a daily log of problems solved, concepts reviewed, and mock interviews completed. Aim for minimum 2 problems daily throughout the job search phase.

Discipline | Consistency | Self-improvement

Resources

PLATFORM **LinkedIn Jobs**

PLATFORM **AngelList (Wellfound) - Startup Jobs**

COMMUNITY **Blind - Anonymous Tech Community**

SUPPORT **Crio.Do Career Support**

Milestone

Receive and accept an offer from a product company with 1.5x+ compensation increase

Your Path Forward

Career Opportunities

- ✓ Software Development Engineer (SDE-1/SDE-2)
- ✓ Backend Developer
- ✓ Full Stack Developer
- ✓ Platform Engineer
- ✓ Senior Software Engineer
- ✓ Software Engineer at FAANG/MAANG

Tips for Success

- Consistency beats intensity - 2 hours daily trumps 14 hours on weekends
- Understand patterns, don't memorize solutions - interviewers can tell the difference
- Practice explaining your thought process out loud while solving problems
- Your services experience is valuable - frame it as problem-solving, not just delivery
- Referrals significantly increase your chances - network actively on LinkedIn
- Don't wait until you're 100% ready to apply - start with backup companies
- Track your weak areas and dedicate extra time to them
- Join communities (Discord, Reddit, Twitter) for motivation and tips
- Take care of your mental health - rejection is part of the process
- Once you get an offer, negotiate - companies expect it

Next Steps

1. Complete the Foundation Reset phase this week
2. Set up LeetCode account and solve your first 10 problems
3. Choose your interview language and commit to it
4. Join the Crio.Do community for guidance and support
5. Block 2-3 hours daily in your calendar for preparation
6. Find an accountability partner for the journey
7. Start building your network on LinkedIn today

Ready to Start Your Journey?

Everything in this roadmap is free to learn on your own. But if you'd like structured guidance, hands-on projects, and mentor support — we'd love to have you at Crio.Do

Explore Crio.Do

https://crio.do